



American University of Sharjah
College of Engineering
Computer Science & Engineering Department
Introduction to Computer Systems – COE 251L

Instructor: Ms. Donthi Sankalpa

Lab#3 - Simple Arrays & XCHG

Arrays

Arrays are consecutive memory locations used to store data in a structured manner. In x86 assembly, arrays are often byte-sized but can also hold larger data types. They are particularly useful for efficiently accessing and manipulating blocks of data. Arrays allow for seamless data transfer between different memory locations, whether within the same segment or across different segments. Additionally, in x86 systems, memory is stored using the little-endian format, meaning that the least significant byte is stored at the lowest memory address, which is important to understand when handling multi-byte values in arrays.

Array Declaration Example	Explanation
myArray BYTE 10 DUP(0)	Declares an array of 10 bytes, all initialized to 0 using the DUP operator.
myArray BYTE 1, 2, 3, 4, 5	Declares an array of 5 bytes initialized with values 1, 2, 3, 4, and 5.
myArray WORD 1234h, 5678h	Declares an array of 2 words (16-bit values) with values 1234h and 5678h.
myArray DWORD 0FFh, 1AAh, 2BBh	Declares an array of 3 double words (32-bit values) initialized to 0FFh, 1AAh, and 2BBh.
myString BYTE 'Hello, World', 0	Declares a null-terminated string with the text 'Hello, World'. The 0 at the end marks the string's termination.

Constants

In x86 assembly, constants can be declared using different directives such as `=`, `EQU`, and `TEXTEQU`, each serving distinct purposes. These constants allow the use of symbolic names in place of hard-coded values, improving code

readability and maintainability. The = directive defines numeric constants that can be modified later within the code segment, while EQU defines immutable constants for numeric or string values, and TEXTEQU is primarily used for defining symbolic text constants.

Constant Declaration Example	Explanation
myConst = 100	Declares a constant myConst with the value of 100. The = operator assigns numeric constants, which can be modified later in the code segment.
myConst EQU 50	Declares a constant myConst with the value of 50. EQU is used for numeric or string constants, which cannot be modified once defined.
myText TEXTEQU <"Hello, World">	Declares a symbolic text constant myText with the value "Hello, World". TEXTEQU is used for defining text constants and cannot be changed after being defined.

In x86 assembly, it is possible to calculate the size of an array immediately after its declaration within the data segment. This can be done using the = directive in combination with the (\$ - array) expression to determine the size in bytes. However, this calculation must be performed right after the array declaration because the \$ symbol represents the current location counter, which changes as new data is declared. Calculating the size later in the code segment could result in incorrect values. For WORD and DWORD arrays, you must still divide by 2 and 4, respectively, to get the correct number of elements.

Sample code:

```
INCLUDE Irvine32.inc
```

```
.data
```

```
myByteArray BYTE 1, 2, 3, 4, 5      ; Declares an array of 5 bytes
sizeofByteArray = ($ - myByteArray) ; Must be calculated right after
declaration (size is 5 bytes)

myWordArray WORD 1234h, 5678h, 9ABCh ; Declares an array of 3 words
sizeofWordArray = ($ - myWordArray) / 2; Must be calculated right after
declaration (size is 3 words)
```

```

myDwordArray DWORD 0FFh, 1AAh, 2BBh    ; Declares an array of 3 double words
sizeofDwordArray = ($ - myDwordArray) / 4; Must be calculated right after
declaration (size is 3 dwords)

```

```

myString BYTE 'Hello, World', 0        ; Declares a string with a null
terminator
sizeofString = ($ - myString)           ; Must be calculated right after
declaration (size is 13 bytes including the null character)

```

```

.code
main PROC
    ; CODE GOES HERE
    exit                ; Exit the program
main ENDP
END main

```

Some Useful Instructions for the Lab

Instruction	Description	Example
OFFSET	Retrieves the memory address (offset) of a variable or label.	MOV EDX, OFFSET myVar ; Loads the offset of myVar into EBX
MOVZX	Moves a value into a register, extending the size with zeroes for unsigned values.	MOVZX EAX, BYTE PTR [EBX] ; Zero-extends an 8-bit value into EAX
MOVSX	Moves a value into a register, extending the size with sign extension for signed values.	MOVSX EAX, BYTE PTR [EBX] ; Sign-extends an 8-bit value into EAX
NEG	Negates (inverts) the value of a register or memory location (two's complement).	NEG EAX ; Negates the value in EAX (if EAX = 5, result is -5)
XCHG	Exchanges the values of two registers or between a register and a memory location.	XCHG EAX, EBX ; Exchanges the values of EAX and EBX

Useful Irvine Commands

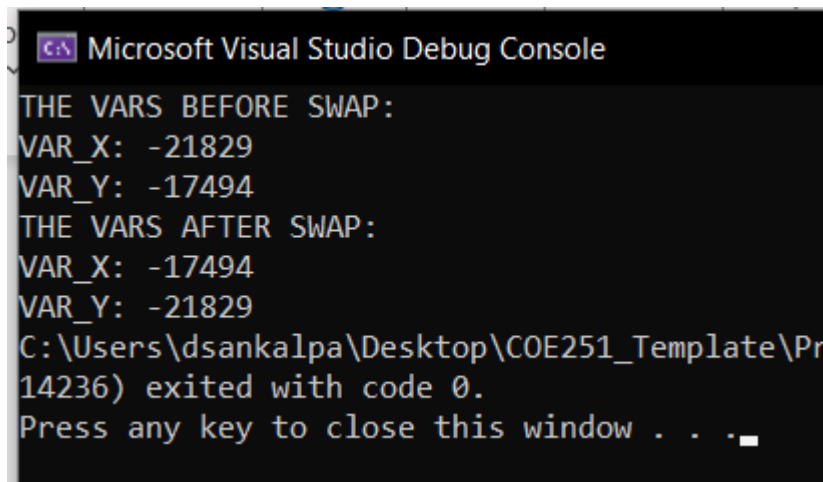
Command	Description	Example
WriteString	Outputs a null-terminated string to the console. You must load the address of the string into EDX before calling this function.	MOV EDX, OFFSET myString CALL WriteString ; Displays the string at myString
ReadInt	Reads an integer input from the console and stores it in EAX.	CALL ReadInt ; Reads an integer from input and stores it in EAX
WriteInt	Outputs the integer in EAX to the console as a decimal value.	MOV EAX, 123 CALL WriteInt ; Displays 123 to the console

Lab Work (Questions 1-5):

NOTE: For all the questions, you will need to initialize memory locations and/or registers before executing your command. Make sure that data variables are declared of the correct size. For example, if you have a count variable going till 10 declare it as a byte and then use the appropriate MOVs.

1. Initialize two-word variables VAR_X and VAR_Y with the values 0AABBh and 0BBAAh respectively. Write an assembly program to swap the two values using XCHG and print the before and after to the user. You must use appropriate move instructions.

Your output should look similar to the following:



```
Microsoft Visual Studio Debug Console
THE VARS BEFORE SWAP:
VAR_X: -21829
VAR_Y: -17494
THE VARS AFTER SWAP:
VAR_X: -17494
VAR_Y: -21829
C:\Users\dsankalpa\Desktop\COE251_Template\Pr
14236) exited with code 0.
Press any key to close this window . . .
```

Solution:

```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
    VAR_X WORD 0AABBh
    VAR_Y WORD 0BBAAh
    string1 BYTE "THE VARS BEFORE SWAP:", 0
    string2 BYTE "VAR_X: ", 0
    string3 BYTE "VAR_Y: ", 0
    string4 BYTE "THE VARS AFTER SWAP:", 0

.code
main PROC
    mov edx, OFFSET string1
    call WriteString
    call crlf

    mov edx, OFFSET string2
    call WriteString
    movsx eax, VAR_X
    call WriteInt
    call crlf
```

```

mov edx, OFFSET string3
call WriteString
movsx eax, VAR_Y
call WriteInt
call crlf

mov edx, OFFSET string4
call WriteString
call crlf

mov ax, VAR_X
XCHG ax, VAR_Y
mov VAR_X, ax

mov edx, OFFSET string2
call WriteString
movsx eax, VAR_X
call WriteInt
call crlf

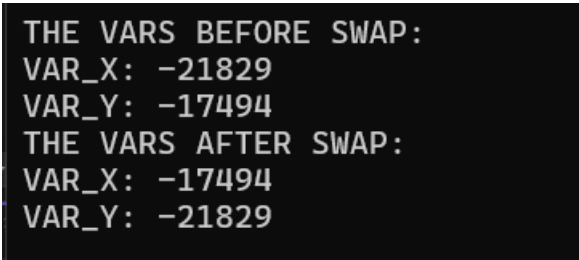
mov edx, OFFSET string3
call WriteString
movsx eax, VAR_Y
call WriteInt
call crlf

exit
main ENDP

END main

```

Output:

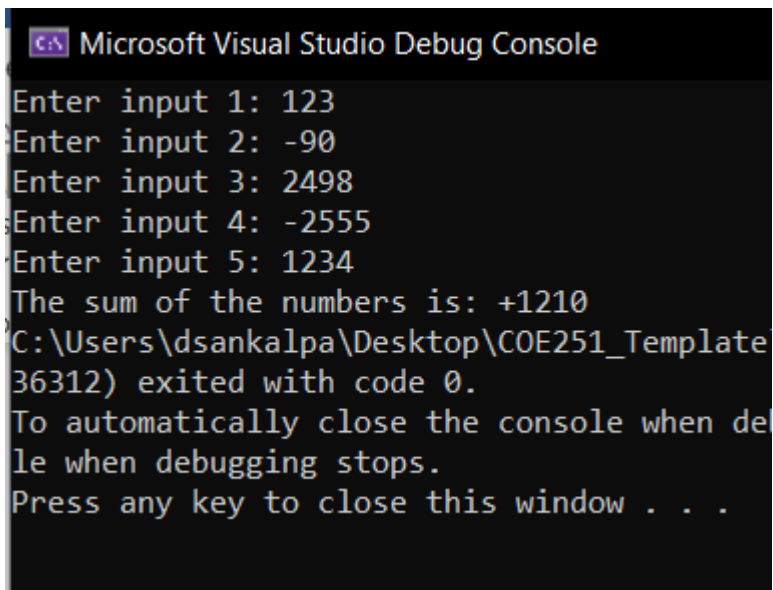


```

THE VARS BEFORE SWAP:
VAR_X: -21829
VAR_Y: -17494
THE VARS AFTER SWAP:
VAR_X: -17494
VAR_Y: -21829

```

2. Write a program that asks the user to input 5 integers and prints out the sum of the five numbers. Your output should be similar as follows:

A screenshot of the Microsoft Visual Studio Debug Console window. The window has a title bar with the Visual Studio logo and the text "Microsoft Visual Studio Debug Console". The console output shows the following text: "Enter input 1: 123", "Enter input 2: -90", "Enter input 3: 2498", "Enter input 4: -2555", "Enter input 5: 1234", "The sum of the numbers is: +1210", "C:\Users\dsankalpa\Desktop\COE251_Template\36312) exited with code 0.", "To automatically close the console when debugging stops.", and "Press any key to close this window . . .".

```
Microsoft Visual Studio Debug Console
Enter input 1: 123
Enter input 2: -90
Enter input 3: 2498
Enter input 4: -2555
Enter input 5: 1234
The sum of the numbers is: +1210
C:\Users\dsankalpa\Desktop\COE251_Template\36312) exited with code 0.
To automatically close the console when debugging stops.
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
string1 BYTE "Enter input 1: ", 0
string2 BYTE "Enter input 2: ", 0
string3 BYTE "Enter input 3: ", 0
string4 BYTE "Enter input 4: ", 0
string5 BYTE "Enter input 5: ", 0
string6 BYTE "The sum of the numbers is: ", 0
sum DWORD 0
```

.code

main PROC

```
mov ecx, 5
```

```
mov edx, OFFSET string1
call WriteString
mov eax, ReadInt
call ReadInt
```

```
add sum, eax
```

```
mov edx, OFFSET string2
call WriteString
mov eax, ReadInt
call ReadInt
```

```
add sum, eax
```

```
mov edx, OFFSET string3
call WriteString
mov eax, ReadInt
call ReadInt
```

```
add sum, eax
```

```

mov edx, OFFSET string4
call WriteString
mov eax, ReadInt
call ReadInt

add sum, eax

mov edx, OFFSET string5
call WriteString
mov eax, ReadInt
call ReadInt

add sum, eax

mov edx, OFFSET string6
call WriteString
mov eax, sum
call WriteInt

call crlf

exit
main ENDP

```

END main

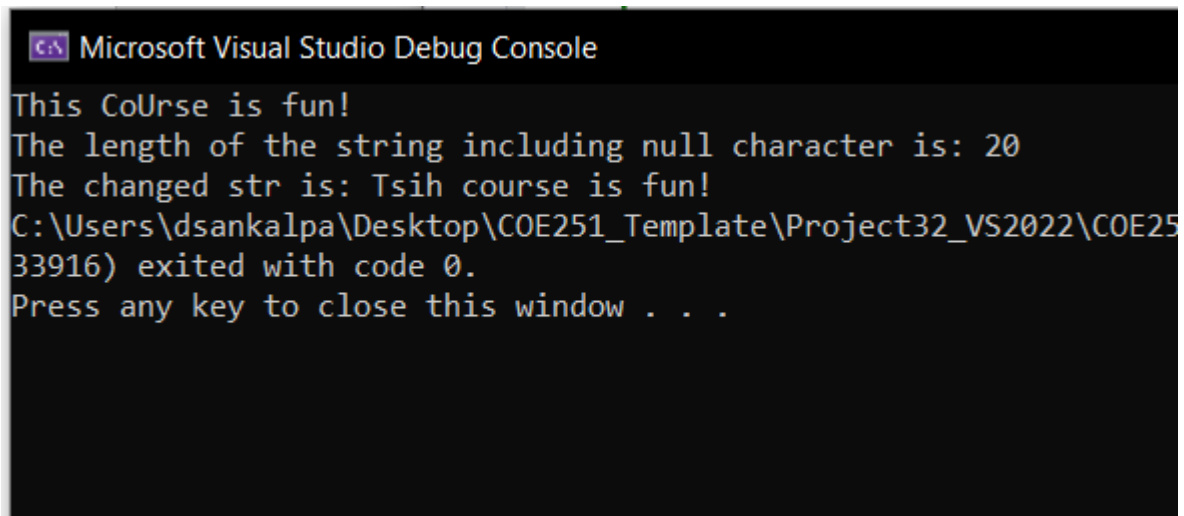
Output:

```

Enter input 1: 123
Enter input 2: -90
Enter input 3: 2498
Enter input 4: -2555
Enter input 5: 1234
The sum of the numbers is: +1210

```

3. Initialize a variable named **var_str** with the value “This CoUrse is fun!”. Then do the following questions:
 - Print the length of the string
 - Write instructions to uncapitalize the 6th and 8th letter. Note: In ASCII, you can change a capital letter to a small letter by adding 32.
 - Write instructions to swap the 2nd and 4th letter.

A screenshot of the Microsoft Visual Studio Debug Console window. The title bar reads "Microsoft Visual Studio Debug Console". The console output shows the following text: "This CoUrse is fun!", "The length of the string including null character is: 20", "The changed str is: Tsih course is fun!", "C:\Users\dsankalpa\Desktop\COE251_Template\Project32_VS2022\COE25133916) exited with code 0.", and "Press any key to close this window . . .".

```
Microsoft Visual Studio Debug Console

This CoUrse is fun!
The length of the string including null character is: 20
The changed str is: Tsih course is fun!
C:\Users\dsankalpa\Desktop\COE251_Template\Project32_VS2022\COE25133916) exited with code 0.
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
var_str BYTE "This CoUrse is fun!", 0
str_len = ($ - var_str)
string2 BYTE "The length of the string including null character is: ", 0
string3 BYTE "The changed str is: ", 0
```

.code

main PROC

```
mov edx, OFFSET var_str
call WriteString
call crlf
```

```
mov edx, OFFSET string2
call WriteString
mov eax, str_len
call WriteDec
call crlf
```

```
mov al, var_str + 5
add al, 32
mov var_str + 5, al
```

```
mov al, var_str + 7
add al, 32
mov var_str + 7, al
```

```
mov al, var_str + 1
mov bl, var_str + 3
mov var_str + 1, bl
mov var_str + 3, al
```

```
mov edx, OFFSET string3
call WriteString
mov edx, OFFSET var_str
call WriteString
call crlf
```

exit

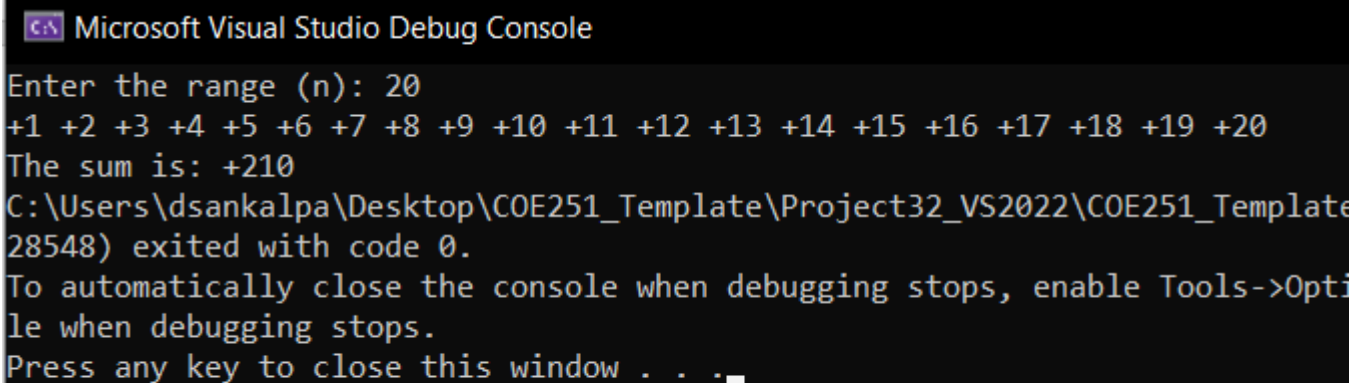
```
main ENDP

END main
```

Output:

```
This CoUrse is fun!
The length of the string including null character is: 20
The changed str is: Tsih course is fun!
```

4. Write a program to ask the user for a range n and then print all the numbers from 1 to n followed by the sum.



```
Microsoft Visual Studio Debug Console

Enter the range (n): 20
+1 +2 +3 +4 +5 +6 +7 +8 +9 +10 +11 +12 +13 +14 +15 +16 +17 +18 +19 +20
The sum is: +210
C:\Users\dsankalpa\Desktop\COE251_Template\Project32_VS2022\COE251_Template
28548) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Opti
le when debugging stops.
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

```
.data
    string1 BYTE "Enter the range (n): ", 0
    string2 BYTE " ", 0
    string3 BYTE "The sum is: ", 0
    sum DWORD 0
```

```
.code
main PROC
    mov edx, OFFSET string1
    call WriteString
    mov eax, ReadDec
    call ReadDec
```

```
    mov ecx, eax
    mov eax, 1
```

```
again:
    call WriteInt
```

```
    mov edx, OFFSET string2
```

```

call WriteString

add sum, eax
add eax, 1

loop again

call crlf

mov edx, OFFSET string3
call WriteString
mov eax, sum
call WriteInt
call crlf

exit
main ENDP

END main

```

Output:

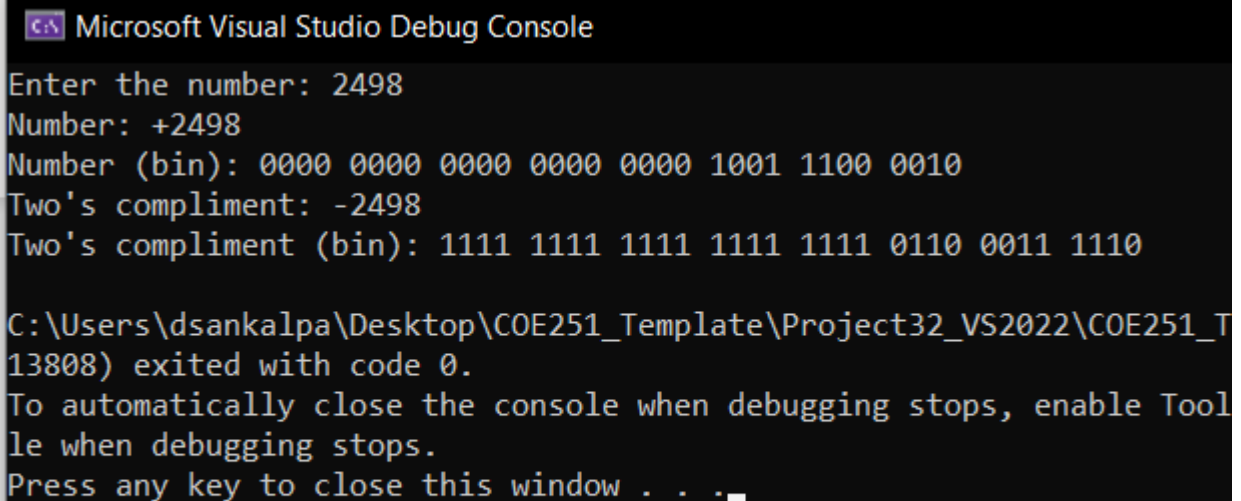
```

Enter the range (n): 20
+1 +2 +3 +4 +5 +6 +7 +8 +9 +10 +11 +12 +13 +14 +15 +16 +17 +18 +19 +20
The sum is: +210

```

5. Write a program that accepts an integer from the user:

- Print the number and its binary form
- Print the 2's compliment and its binary form



```

Microsoft Visual Studio Debug Console

Enter the number: 2498
Number: +2498
Number (bin): 0000 0000 0000 0000 0000 1001 1100 0010
Two's compliment: -2498
Two's compliment (bin): 1111 1111 1111 1111 1111 0110 0011 1110

C:\Users\dsankalpa\Desktop\COE251_Template\Project32_VS2022\COE251_T
13808) exited with code 0.
To automatically close the console when debugging stops, enable Tool
le when debugging stops.
Press any key to close this window . . .

```

Solution:

```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
    string1 BYTE "Enter the number: ", 0
    string2 BYTE "Number: ", 0
    string3 BYTE "Number (bin): ", 0
    string4 BYTE "Two's compliment: ", 0
    string5 BYTE "Two's compliment (bin): ", 0

.code
main PROC
    mov edx, OFFSET string1
    call WriteString
    mov eax, ReadDec
    call ReadDec

    mov edx, OFFSET string2
    call WriteString
    call WriteInt
    call crlf

    mov edx, OFFSET string3
    call WriteString
    call WriteBin
    call crlf

    mov edx, OFFSET string4
    call WriteString
    neg eax
    call WriteInt
    call crlf

    mov edx, OFFSET string5
    call WriteString
    call WriteBin
    call crlf

exit
main ENDP

END main
```

Output:

```
Enter the number: 2498
Number: +2498
Number (bin): 0000 0000 0000 0000 0000 1001 1100 0010
Two's compliment: -2498
Two's compliment (bin): 1111 1111 1111 1111 1111 0110 0011 1110
```